



# Seek & Destroy

SQL Performance Playground — learn Microsoft SQL Server performance tuning by doing.

User Manual · 80 lessons across four levels · Updated 30 July 2026

---

## Contents

1. What Seek & Destroy is
2. Getting started
3. The interface
4. Working through a lesson
5. Running a single statement (right-click)
6. Reading the results
7. Hints, solutions and the AI tutor
8. Concurrency lessons
9. Settings
10. Customising the workspace
11. Keyboard shortcuts
12. The curriculum
13. Troubleshooting

## 1. What Seek & Destroy is

---

Seek & Destroy is an interactive course in SQL Server performance tuning. Rather than reading about slow queries, you fix them: every lesson gives you a genuinely slow query running against a **real SQL Server 2022 engine**, with real data volumes, real execution plans, real logical-read counts and — in the concurrency lessons — real blocking and real deadlocks.

Your work is **auto-graded**. Each lesson defines pass conditions expressed in terms the engine can prove: which physical operators must (or must not) appear in the plan, how many logical reads a

table may cost, whether your rewritten query still returns the same rows. A lesson counts as solved only when the engine's own measurements say you fixed it.

**Nothing is simulated.** The plans you read are captured with `SET STATISTICS XML ON` from the query you actually ran, and the read counts come from `SET STATISTICS IO ON`. When a lesson says a fix took a query from 1,036 reads to 3, those are measured numbers.

## 2. Getting started

### Use the hosted version

The playground runs in your browser at `salmon-field-04fa1fd10.7.azurestaticapps.net`. Nothing to install.

### Or run it locally with Docker

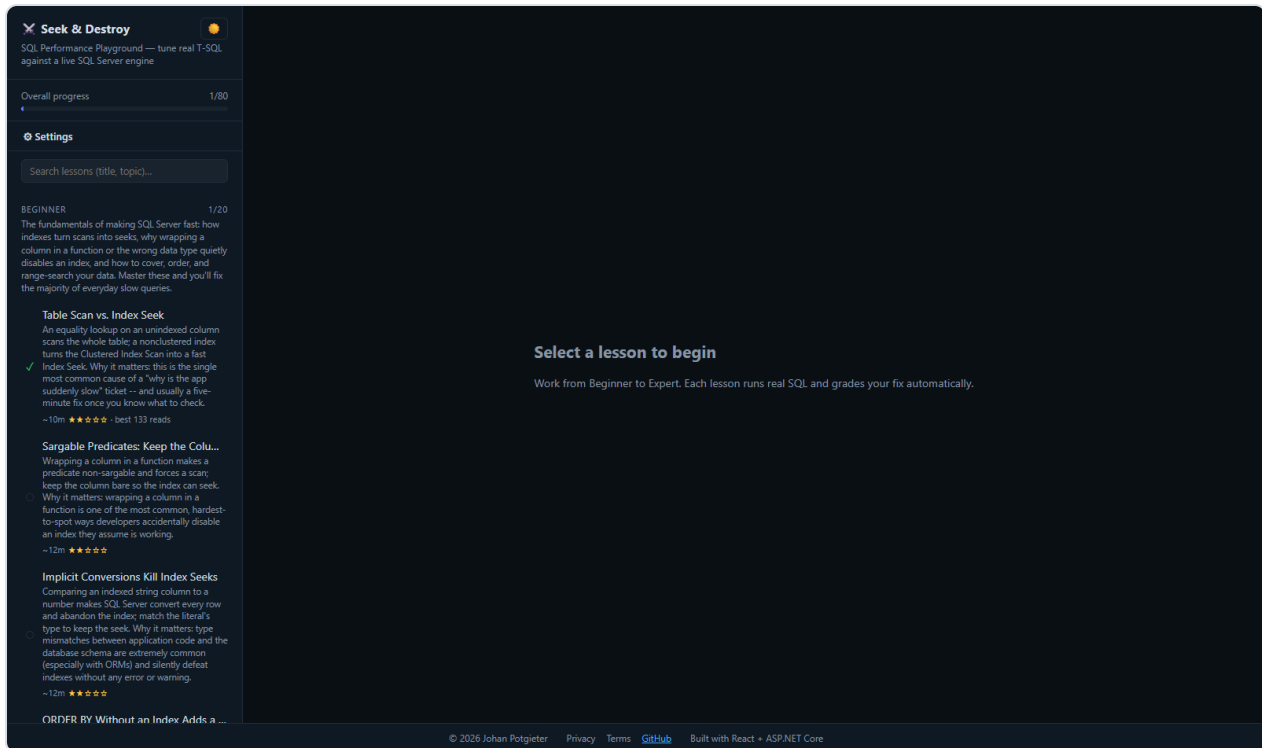
```
git clone https://github.com/johanccs/seek-and-destroy-sql
cd seek-and-destroy-sql
docker compose up -d
```

Then open `http://localhost:5173`. Three containers start: the web front end, the API, and SQL Server 2022.

| Service    | Address                                                                  |
|------------|--------------------------------------------------------------------------|
| Web app    | <code>http://localhost:5173</code>                                       |
| API        | <code>http://localhost:5080</code> (health at <code>/api/health</code> ) |
| SQL Server | <code>localhost,14330</code> — user <code>sa</code>                      |

SQL Server is published on port **14330**, not the default 1433, so it will not clash with any SQL Server you already run.

### 3. The interface



*The lesson navigator. Levels group the curriculum; each lesson shows a description, estimated time, a difficulty rating and your best result.*

The left sidebar is the navigator. It shows your overall progress, a search box that filters by title, description or topic, and the four levels. A green tick marks a solved lesson, and solved lessons display your best logical-read count so you can try to beat it.

**SQL Performance Playground** — tune real T-SQL against a live SQL Server engine

Overall progress 1/80

Settings

Search lessons (title, topic)...

BEGINNER 1/20

The fundamentals of making SQL Server fast: how indexes turn scans into seeks, why wrapping a column in a function or the wrong data type quietly disables an index, and how to cover, order, and range-search your data. Master these and you'll fix the majority of everyday slow queries.

**Table Scan vs. Index Seek**

An equality lookup on an unindexed column scans the whole table; a non-clustered index turns the Clustered Index Scan into a fast Index Seek. Why it matters: this is the single most common cause of a "why is the app suddenly slow" ticket -- and usually a five-minute fix once you know what to check.

~10m ★★★★★ - best 133 reads

**Sargable Predicates: Keep the Columns Simple**

Wrapping a column in a function makes a predicate non-sargable and forces a scan; keep the column bare so the index can seek. Why it matters: wrapping a column in a function is one of the most common, hardest-to-spot ways developers accidentally disable an index they assume is working.

~12m ★★★★★

**Implicit Conversions Kill Index Seeks**

Comparing an indexed string column to a number makes SQL Server convert every row and abandon the index; match the literal's type to keep the seek. Why it matters: type mismatches between application code and the database schema are extremely common (especially with ORM) and silently defeat indexes without any error or warning.

~12m ★★★★★

ORDER BY Without an Index Adirix a ...

## Table Scan vs. Index Seek

indexing plan-reading ~10 min ★★★★★ solved

Sarge — AI Tutor ▶

LESSON A- 14ps A+

▶ Run ⚙️ Reset Lesson 💡 Hint (0/3)

Show Solution A- 13ps A+

### What This Lesson Is About

This lesson is about the single most common performance problem in any database: asking for a few rows but making the engine look at *all* of them. You'll see the difference between a **table scan** (checking every row, one by one) and an **index seek** (jumping straight to the rows you need).

Why does this happen? A table on its own is just a big pile of rows with no particular fast way to find "the ones where CustomerId = 42." Unless you build a helper structure, the database has no shortcut — it has to open every page and check every row itself.

Here's the mental model: think of a huge phone book with no alphabetical order — names just written down in the order people signed up. If you want everyone named "Smith," you have to read every single page, front to back. That's a table scan.

Now imagine someone re-sorts a copy of that phone book by last name and tapes labels on it. Suddenly you can flip straight to the "S" section. That's what an **index** does — it's a separate, sorted copy of some columns that lets SQL Server jump straight to the matching rows instead of reading everything. This lesson shows you how to spot the "reading everything" problem in a query plan, and how adding the right index turns a slow scan into a fast, targeted seek.

### Scenario

An order-lookup screen runs `SELECT * FROM Orders WHERE CustomerId = 42` on a 200,000-row table. It feels sluggish and gets worse as the table grows.

### Why it's slow

Run a query to see results, statistics and the execution plan.

```
1 SELECT * FROM Orders WHERE CustomerId = 42;
```

© 2026 Johan Potgieter Privacy Terms GitHub Built with React + ASP.NET Core

*A lesson: explanation on the left, SQL editor top right, results underneath.*

Opening a lesson gives you three regions: the **narrative** explaining the problem, the **SQL editor** holding the starting query, and the **results area** below it. All three are resizable — see section 10.

## 4. Working through a lesson

Every lesson follows the same rhythm:

1. Read the narrative. It describes a realistic scenario and explains why the query is slow.
2. Run the starting query as-is to see the damage — the plan and the read count.
3. Rewrite the query, or add an index, to fix it.
4. Run again. The grader tells you which conditions now pass.

### Seek & Destroy

SQL Performance Playground — tune real T-SQL against a live SQL Server engine

Overall progress 1/80

**Settings**

Search lessons (title, topic)...

BEGINNER 1/20

The fundamentals of making SQL Server fast: how indexes turn scans into seeks, why wrapping a column in a function or the wrong data type quietly disables an index, and how to cover, order, and range-search your data. Master these and you'll fix the majority of everyday slow queries.

### Table Scan vs. Index Seek

An equality lookup on an unindexed column scans the whole table; a nonclustered index turns the Clustered Index Scan into a fast Index Seek. Why it matters: this is the single most common cause of a "why is the app suddenly slow?" ticket — and usually a five-minute fix once you know what to check.  
~10m ★★★★★ - best 133 reads

### Sargable Predicates: Keep the Columns Simple

Wrapping a column in a function makes a predicate non-sargable and forces a scan; keep the column bare so the index can seek. Why it matters: wrapping a column in a function is one of the most common, hardest-to-spot ways developers accidentally disable an index they assume is working.  
~12m ★★★★★

### Implicit Conversions Kill Index Seeks

Comparing an indexed string column to a number makes SQL Server convert every row and abandon the index; match the literal's type to keep the seek. Why it matters: type mismatches between application code and the database schema are extremely common (especially with ORMs) and silently defeat indexes without any error or warning.  
~12m ★★★★★

ORDER BY Without an Index Adds a ...

## Table Scan vs. Index Seek

indexing plan-reading ~10 min ★★★★★ · solved

Sarge — AI Tutor ▶

LESSON A- 14px A+

▶ Run ⚙ Reset Lesson 💡 Hint (1/3)

Show Solution A- 13px A+

### What This Lesson Is About

This lesson is about the single most common performance problem in any database: asking for a few rows but making the engine look at *all* of them. You'll see the difference between a **table scan** (checking every row, one by one) and an **index seek** (jumping straight to the rows you need).

Why does this happen? A table on its own is just a big pile of rows with no particular fast way to find "the ones where CustomerId = 42." Unless you build a helper structure, the database has no shortcut — it has to open every page and check every row itself.

Here's the mental model: think of a huge phone book with no alphabetical order — names just written down in the order people signed up. If you want everyone named "Smith," you have to read every single page, front to back. That's a table scan.

Now imagine someone re-sorts a copy of that phone book by last name and tapes labels on it. Suddenly you can flip straight to the "S" section. That's what an **index** does — it's a separate, sorted copy of some columns that lets SQL Server jump straight to the matching rows instead of reading everything. This lesson shows you how to spot the "reading everything" problem in a query plan, and how adding the right index turns a slow scan into a fast, targeted seek.

✓ All conditions pass.

- ✓ Plan uses Index Seek — node 2
- ✓ No Clustered Index Scan in plan — not present
- ✓ Logical reads on Orders ≤ 200 — 133 reads
- ✓ Result matches baseline — identical

### Scenario

Your order-lookup screen runs `SELECT * FROM Orders WHERE CustomerId = 42` on a 200,000-row table. It feels sluggish and gets worse as the table grows.

### Why it's slow

| OrderId | CustomerId | OrderDate                   | Total | Status  |
|---------|------------|-----------------------------|-------|---------|
| 41      | 42         | 2024-11-21T00:00:00.0000000 | 61.5  | Closed  |
| 5041    | 42         | 2024-03-11T00:00:00.0000000 | 61.5  | Shipped |
| 10041   | 42         | 2024-06-20T00:00:00.0000000 | 61.5  | Open    |

Results Statistics Execution Plan Messages A- 13px A+

© 2026 Johan Potgieter Privacy Terms GitHub Built with React + ASP.NET Core

*A passing run. Every pass condition is listed with the measured evidence beside it.*

The banner lists each condition individually with the evidence — *Plan uses Index Seek — node 2, Logical reads on Orders ≤ 200 — 133 reads*. When a run fails, the same list shows exactly which conditions did not hold, so you always know what is left to fix.

Press `Ctrl` + `Enter` to run, or click **Run**. This is the graded run: it evaluates your whole editor contents and can mark the lesson solved.

## 5. Running a single statement (right-click)

A solution is often more than one statement — typically an index build followed by the query you are tuning:

```
CREATE NONCLUSTERED INDEX IX_Orders_CustomerId ON Orders(CustomerId);
SELECT * FROM Orders WHERE CustomerId = 42;
```

Once the index exists, you usually want to re-run only the `SELECT`. Select that statement, right-click it, and choose **Run Selection (not graded)**.

Seek & Destroy

SQL Performance Playground — tune real T-SQL against a live SQL Server engine

Overall progress5/80

Settings

Search lessons (title, topic)...

BEGINNER5/20

The fundamentals of making SQL Server fast: how indexes turn scans into seeks, why wrapping a column in a function or the wrong data type quietly disables an index, and how to cover, order, and range-search your data. Master these and you'll fix the majority of everyday slow queries.

Table Scan vs. Index Seek

An equality lookup on an unindexed column scans the whole table; a nonclustered index turns the Clustered Index Scan into a fast Index Seek. Why it matters: this is the single most common cause of a "why is the app suddenly slow" ticket — and usually a five-minute fix once you know what to check.  
~10m ★★★★★ · best 133 reads

Sargable Predicates: Keep the Colu...

Wrapping a column in a function makes a predicate non-sargable and forces a scan; keep the column bare so the index can seek. Why it matters: wrapping a column in a function is one of the most common, hardest-to-spot ways developers accidentally disable an index they assume is working.  
~12m ★★★★★ · best 165 reads

Implicit Conversions Kill Index Seeks

Comparing an indexed string column to a number makes SQL Server convert every row and abandon the index; match the literal's type to keep the seek. Why it matters: type mismatches between application code and the database schema are extremely common (especially with ORM) and silently defeat indexes without any error or warning.  
~12m ★★★★★

ORDER BY Without an Index Adds a ...

Table Scan vs. Index Seek

indexing · plan-reading · ~10 min ★★★★★ · solved

Sarge — AI Tutor

LESSON

A- 14px A+

What This Lesson Is About

This lesson is about the single most common performance problem in any database: asking for a few rows but making the engine look at all of them. You'll see the difference between a **table scan** (checking every row, one by one) and an **index seek** (jumping straight to the rows you need).

Why does this happen? A table on its own is just a big pile of rows with no particular fast way to find "the ones where CustomerId = 42." Unless you build a helper structure, the database has no shortcut — it has to open every page and check every row itself.

Here's the mental model: think of a huge phone book with no alphabetical order — names just written down in the order people signed up. If you want everyone named "Smith," you have to read every single page, front to back. That's a table scan.

Now imagine someone re-sorts a copy of that phone book by last name and tapes labels on it. Suddenly you can flip straight to the "S" section. That's what an **index** does — it's a separate, sorted copy of some columns that lets SQL Server jump straight to the matching rows instead of reading everything. This lesson shows you how to spot the "reading everything" problem in a query plan, and how adding the right index turns a slow scan into a fast, targeted seek.

Scenario

An order-lookup screen runs **SELECT \* FROM Orders WHERE CustomerId = 42** on a 200,000-row table. It feels sluggish and gets worse as the table grows.

Run

Reset Lesson

Hint (0/3)

Show Solution

A- 13px A+

1 SELECT \* FROM Orders WHERE CustomerId = 42;

Run Selection (not graded)

Change All Occurrences Ctrl+F2

Cut

Copy

Paste

Command Palette F1

Run a query to see results, statistics and the execution plan.

© 2026 Johan Potgieter · Privacy · Terms · GitHub · Built with React + ASP.NET Core

Select a statement, right-click, and Run Selection (not graded) appears at the top of the editor's context menu.

Only the highlighted text is sent to the engine. You get its results, its statistics and its execution plan exactly as for a full run.

Seek & Destroy

SQL Performance Playground — tune real T-SQL against a live SQL Server engine

Overall progress1/80

Settings

Search lessons (title, topic)...

BEGINNER1/20

The fundamentals of making SQL Server fast: how indexes turn scans into seeks, why wrapping a column in a function or the wrong data type quietly disables an index, and how to cover, order, and range-search your data. Master these and you'll fix the majority of everyday slow queries.

Table Scan vs. Index Seek

An equality lookup on an unindexed column scans the whole table; a nonclustered index turns the Clustered Index Scan into a fast Index Seek. Why it matters: this is the single most common cause of a "why is the app suddenly slow" ticket — and usually a five-minute fix once you know what to check.  
~10m ★★★★★ · best 133 reads

Sargable Predicates: Keep the Colu...

Wrapping a column in a function makes a predicate non-sargable and forces a scan; keep the column bare so the index can seek. Why it matters: wrapping a column in a function is one of the most common, hardest-to-spot ways developers accidentally disable an index they assume is working.  
~12m ★★★★★

Implicit Conversions Kill Index Seeks

Comparing an indexed string column to a number makes SQL Server convert every row and abandon the index; match the literal's type to keep the seek. Why it matters: type mismatches between application code and the database schema are extremely common (especially with ORM) and silently defeat indexes without any error or warning.  
~12m ★★★★★

ORDER BY Without an Index Adds a ...

Table Scan vs. Index Seek

indexing · plan-reading · ~10 min ★★★★★ · solved

Sarge — AI Tutor

LESSON

A- 14px A+

What This Lesson Is About

This lesson is about the single most common performance problem in any database: asking for a few rows but making the engine look at all of them. You'll see the difference between a **table scan** (checking every row, one by one) and an **index seek** (jumping straight to the rows you need).

Why does this happen? A table on its own is just a big pile of rows with no particular fast way to find "the ones where CustomerId = 42." Unless you build a helper structure, the database has no shortcut — it has to open every page and check every row itself.

Here's the mental model: think of a huge phone book with no alphabetical order — names just written down in the order people signed up. If you want everyone named "Smith," you have to read every single page, front to back. That's a table scan.

Now imagine someone re-sorts a copy of that phone book by last name and tapes labels on it. Suddenly you can flip straight to the "S" section. That's what an **index** does — it's a separate, sorted copy of some columns that lets SQL Server jump straight to the matching rows instead of reading everything. This lesson shows you how to spot the "reading everything" problem in a query plan, and how adding the right index turns a slow scan into a fast, targeted seek.

Scenario

An order-lookup screen runs **SELECT \* FROM Orders WHERE CustomerId = 42** on a 200,000-row table. It feels sluggish and gets worse as the table grows.

Why it's slow

Run

Reset Lesson

Hint (1/3)

Show Solution

A- 13px A+

1 SELECT COUNT(\*) FROM Orders;

2 SELECT \* FROM Orders WHERE CustomerId = 42;

Run Selection (not graded)

Change All Occurrences Ctrl+F2

Cut

Copy

Paste

Command Palette F1

Results

Statistics

Execution Plan

Messages

Scratch run — not graded

A- 13px A+

| OrderId | CustomerId | OrderDate                   | Total | Status  |
|---------|------------|-----------------------------|-------|---------|
| 3       | 4          | 2024-12-29T00:00:00.0000000 | 4.5   | Open    |
| 5803    | 4          | 2024-04-18T00:00:00.0000000 | 4.5   | Closed  |
| 18803   | 4          | 2024-08-06T00:00:00.0000000 | 4.5   | Shipped |
| 15803   | 4          | 2024-11-24T00:00:00.0000000 | 4.5   | Open    |
| 29803   | 4          | 2024-03-14T00:00:00.0000000 | 4.5   | Closed  |
| 25803   | 4          | 2024-07-02T00:00:00.0000000 | 4.5   | Shipped |
| 38803   | 4          | 2024-10-20T00:00:00.0000000 | 4.5   | Open    |
| 35803   | 4          | 2024-02-08T00:00:00.0000000 | 4.5   | Closed  |
| 48803   | 4          | 2024-05-28T00:00:00.0000000 | 4.5   | Shipped |
| 45803   | 4          | 2024-09-15T00:00:00.0000000 | 4.5   | Open    |

© 2026 Johan Potgieter · Privacy · Terms · GitHub · Built with React + ASP.NET Core

A scratch run is marked 'Scratch run — not graded' and deliberately shows no pass/fail verdict.

**A selection run is never graded.** It cannot pass a lesson, cannot fail one, and cannot mark anything solved — the yellow *Scratch run — not graded* badge tells you which kind of run you are looking at. This is deliberate: grading a fragment would be misleading in both directions. A lesson graded on "logical reads on Orders" assumes the query runs last in the batch, so re-running just the `SELECT` after the index already exists would *appear* to pass without your solution containing the index at all. To submit, use **Run**.

With nothing selected, the menu item is not shown at all. Select the text first, then right-click.

## 6. Reading the results

---

Four tabs sit under the editor.

### Results

The rows your query returned, capped at 500 for display.

# Statistics

Seek & Destroy

SQL Performance Playground — tune real T-SQL against a live SQL Server engine

Overall progress

1/80

Settings

Search lessons (title, topic)...

BEGINNER 1/20

The fundamentals of making SQL Server fast: how indexes turn scans into seeks, why wrapping a column in a function or the wrong data type quietly disables an index, and how to cover, order, and range-search your data. Master these and you'll fix the majority of everyday slow queries.

Table Scan vs. Index Seek

An equality lookup on an unindexed column scans the whole table; a nonclustered index turns the Clustered Index Scan into a fast Index Seek. Why it matters: this is the single most common cause of a "why is the app suddenly slow" ticket — and usually a five-minute fix once you know what to check. —10m ★★★★★ · best 133 reads

Sargable Predicates: Keep the Colu...

Wrapping a column in a function makes a predicate non-sargable and forces a scan; keep the column bare so the index can seek. Why it matters: wrapping a column in a function is one of the most common, hardest-to-spot ways developers accidentally disable an index they assume is working. —12m ★★★★★

Implicit Conversions Kill Index Seeks

Comparing an indexed string column to a number makes SQL Server convert every row and abandon the index; match the literal's type to keep the seek. Why it matters: type mismatches between application code and the database schema are extremely common (especially with ORMs) and silently defeat indexes without any error or warning. —12m ★★★★★

ORDER BY Without an Index Adds a ...

Table Scan vs. Index Seek

Indexing · plan-reading · ~10 min ★★★★★ · solved

Sarge — AI Tutor

LESSON A- 14px A+

What This Lesson Is About

This lesson is about the single most common performance problem in any database: asking for a few rows but making the engine look at all of them. You'll see the difference between a **table scan** (checking every row, one by one) and an **index seek** (jumping straight to the rows you need).

Why does this happen? A table on its own is just a big pile of rows with no particular fast way to find "the ones where CustomerId = 42." Unless you build a helper structure, the database has no shortcut — it has to open every page and check every row itself.

Here's the mental model: think of a huge phone book with no alphabetical order — names just written down in the order people signed up. If you want everyone named "Smith," you have to read every single page, front to back. That's a table scan.

Now imagine someone re-sorts a copy of that phone book by last name and tapes labels on it. Suddenly you can flip straight to the "S" section. That's what an **index** does — it's a separate, sorted copy of some columns that lets SQL Server jump straight to the matching rows instead of reading everything. This lesson shows you how to spot the "reading everything" problem in a query plan, and how adding the right index turns a slow scan into a fast, targeted seek.

Scenario

An order-lookup screen runs `SELECT * FROM Orders WHERE CustomerId = 42` on a 200,000-row table. It feels sluggish and gets worse as the table grows.

Why it's slow

Run Reset Lesson Hint (1/3) Show Solution A- 13px A+

1 SELECT \* FROM Orders WHERE CustomerId = 42;

All conditions pass.

Plan uses Index Seek — node 2

No Clustered Index Scan in plan — not present

Logical reads on Orders ≤ 200 — 133 reads

Result matches baseline — identical

Results Statistics Execution Plan Messages A- 13px A+

LOGICAL READS 133

PHYSICAL READS 0

CPU TIME (MS) 8

ELAPSED (MS) 9

ROWS AFFECTED 0

© 2026 Johan Potgieter Privacy Terms GitHub Built with React + ASP.NET Core

Logical reads, CPU and elapsed time, with the change since your previous run.

Logical reads are the number the lessons care about most: pages the engine had to touch. When you improve a query, the delta against your previous run is shown, so progress is visible immediately.

# Execution Plan

The screenshot displays the 'Seek & Destroy' SQL Performance Playground interface. The top left sidebar shows the course title 'Seek & Destroy', a progress indicator '1/80', and a 'Settings' section with a search bar. The main content area is titled 'Table Scan vs. Index Seek' and includes a 'LESSON' section with a description of the lesson's purpose: to teach the difference between a table scan and an index seek. Below this, a 'Scenario' section describes a query: `SELECT * FROM Orders WHERE CustomerId = 42;`. The right side of the interface shows the 'Run' button, a 'Reset Lesson' button, and a 'Hint (1/3)' button. The bottom right section displays the 'Execution Plan' tab, which shows a tree of physical operators with their relative costs. The plan includes a 'Nested Loops (Inner Join)' operator with a cost of 0.1% and a warning icon.

**Seek & Destroy**  
SQL Performance Playground — tune real T-SQL against a live SQL Server engine

Overall progress 1/80

**Settings**  
Search lessons (title, topic)...

**BEGINNER** 1/20  
The fundamentals of making SQL Server fast: how indexes turn scans into seeks, why wrapping a column in a function or the wrong data type quietly disables an index, and how to cover, order, and range-search your data. Master these and you'll fix the majority of everyday slow queries.

**Table Scan vs. Index Seek**  
An equality lookup on an unindexed column scans the whole table; a nonclustered index turns the Clustered Index Scan into a fast Index Seek. Why it matters: this is the single most common cause of a "why is the app suddenly slow" ticket — and usually a five-minute fix once you know what to check. ~10m ★★★★★ · best 133 reads

**Sargable Predicates: Keep the Column Bare**  
Wrapping a column in a function makes a predicate non-sargable and forces a scan; keep the column bare so the index can seek. Why it matters: wrapping a column in a function is one of the most common, hardest-to-spot ways developers accidentally disable an index they assume is working. ~12m ★★★★★

**Implicit Conversions Kill Index Seeks**  
Comparing an indexed string column to a number makes SQL Server convert every row and abandon the index; match the literal's type to keep the seek. Why it matters: type mismatches between application code and the database schema are extremely common (especially with ORMs) and silently defeat indexes without any error or warning. ~12m ★★★★★

**ORDER BY Without an Index Adds a ...**

**Table Scan vs. Index Seek**  
Indexing · plan-reading · ~10 min ★★★★★ · solved

Sarge — AI Tutor

LESSON A- 14px A+

**What This Lesson Is About**  
This lesson is about the single most common performance problem in any database: asking for a few rows but making the engine look at all of them. You'll see the difference between a **table scan** (checking every row, one by one) and an **index seek** (jumping straight to the rows you need).  
  
Why does this happen? A table on its own is just a big pile of rows with no particular fast way to find "the ones where CustomerId = 42." Unless you build a helper structure, the database has no shortcut — it has to open every page and check every row itself.  
  
Here's the mental model: think of a huge phone book with no alphabetical order — names just written down in the order people signed up. If you want everyone named "Smith," you have to read every single page, front to back. That's a table scan.  
  
Now imagine someone re-sorts a copy of that phone book by last name and tapes labels on it. Suddenly you can flip straight to the "S" section. That's what an **index** does — it's a separate, sorted copy of some columns that lets SQL Server jump straight to the matching rows instead of reading everything. This lesson shows you how to spot the "reading everything" problem in a query plan, and how adding the right index turns a slow scan into a fast, targeted seek.

**Scenario**  
An order-lookup screen runs `SELECT * FROM Orders WHERE CustomerId = 42` on a 200,000-row table. It feels sluggish and gets worse as the table grows.

**Why it's slow**

Run Reset Lesson Hint (1/3) Show Solution A- 13px A+

1 `SELECT * FROM Orders WHERE CustomerId = 42;`

**All conditions pass.**  
✓ Plan uses Index Seek — node 2  
✓ No Clustered Index Scan in plan — not present  
✓ Logical reads on Orders ≤ 200 — 133 reads  
✓ Result matches baseline — identical

Results Statistics **Execution Plan** Messages A- 13px A+

**Nested Loops (Inner Join)**  
rows: est 40 / actual 40 · cost 0.1%

© 2026 Johan Potgieter Privacy Terms [Github](#) Built with React + ASP.NET Core

The captured actual execution plan as a readable operator tree, with cost shares and warnings.

This is the *actual* plan for the query you ran, shown as a tree of physical operators with their relative costs. Learning to spot *Clustered Index Scan* where you wanted *Index Seek* is most of the skill this course teaches. Plan warnings — implicit conversions, tempdb spills, missing-index suggestions — are surfaced here too, with a dot on the tab when present.

## Messages

Raw engine messages, including the full `STATISTICS IO` output.

# 7. Hints, solutions and the AI tutor

✕ Seek & Destroy

SQL Performance Playground — tune real T-SQL against a live SQL Server engine

Overall progress 1/80

Settings

Search lessons (title, topic)...

BEGINNER 1/20

The fundamentals of making SQL Server fast: how indexes turn scans into seeks, why wrapping a column in a function or the wrong data type quietly disables an index, and how to cover, order, and range-search your data. Master these and you'll fix the majority of everyday slow queries.

Table Scan vs. Index Seek

An equality lookup on an unindexed column scans the whole table; a nonclustered index turns the Clustered Index Scan into a fast Index Seek. Why it matters: this is the single most common cause of a "why is the app suddenly slow" ticket -- and usually a five-minute fix once you know what to check. ~10m ★★★★★ · best 133 reads

Sargable Predicates Keep the Columns Sargable

Wrapping a column in a function makes a predicate non-sargable and forces a scan; keep the column bare so the index can seek. Why it matters: wrapping a column in a function is one of the most common, hardest-to-spot ways developers accidentally disable an index they assume is working. ~12m ★★★★★

Implicit Conversions Kill Index Seeks

Comparing an indexed string column to a number makes SQL Server convert every row and abandon the index; match the literal's type to keep the seek. Why it matters: type mismatches between application code and the database schema are extremely common (especially with ORMs) and silently defeat indexes without any error or warning. ~12m ★★★★★

ORDER BY Without an Index Adds a ...

Table Scan vs. Index Seek

Indexing · plan-reading · ~10 min ★★★★★ · solved

Sarge — AI Tutor

LESSON A- 14px A+

What This Lesson Is About

This lesson is about the single most common performance problem in any database: asking for a few rows but making the engine look at *all* of them. You'll see the difference between a **table scan** (checking every row, one by one) and an **index seek** (jumping straight to the rows you need).

Why does this happen? A table on its own is just a big pile of rows with no particular fast way to find "the ones where CustomerId = 42." Unless you build a helper structure, the database has no shortcut — it has to open every page and check every row itself.

Here's the mental model: think of a huge phone book with no alphabetical order — names just written down in the order people signed up. If you want everyone named "Smith," you have to read every single page, front to back. That's a table scan.

Now imagine someone re-sorts a copy of that phone book by last name and tapes labels on it. Suddenly you can flip straight to the "S" section. That's what an **index** does — it's a separate, sorted copy of some columns that lets SQL Server jump straight to the matching rows instead of reading everything. This lesson shows you how to spot the "reading everything" problem in a query plan, and how adding the right index turns a slow scan into a fast, targeted seek.

Scenario

An order-lookup screen runs `SELECT * FROM Orders WHERE CustomerId = 42` on a 200,000-row table. It feels sluggish and gets worse as the table grows.

Why it's slow

Run Reset Lesson Hint (2/3) Show Solution A- 13px A+

1 `SELECT COUNT(*) FROM Orders;`  
2 `SELECT * FROM Orders WHERE CustomerId = 42;`

Results Statistics Execution Plan Messages Scratch run — not graded A- 13px A+

| OrderId | CustomerId | OrderDate                   | Total | Status  |
|---------|------------|-----------------------------|-------|---------|
| 3       | 4          | 2024-12-29T00:00:00.0000000 | 4.5   | Open    |
| 5003    | 4          | 2024-04-10T00:00:00.0000000 | 4.5   | Closed  |
| 10003   | 4          | 2024-08-06T00:00:00.0000000 | 4.5   | Shipped |
| 15003   | 4          | 2024-11-24T00:00:00.0000000 | 4.5   | Open    |
| 20003   | 4          | 2024-03-14T00:00:00.0000000 | 4.5   | Closed  |
| 25003   | 4          | 2024-07-02T00:00:00.0000000 | 4.5   | Shipped |
| 30003   | 4          | 2024-10-20T00:00:00.0000000 | 4.5   | Open    |
| 35003   | 4          | 2024-02-08T00:00:00.0000000 | 4.5   | Closed  |
| 40003   | 4          | 2024-05-28T00:00:00.0000000 | 4.5   | Shipped |
| 45003   | 4          | 2024-09-15T00:00:00.0000000 | 4.5   | Open    |

© 2026 Johan Potgieter Privacy Terms GitHub Built with React + ASP.NET Core

Hints reveal one at a time, so you can take only as much help as you need.

Each lesson carries up to three hints, revealed one at a time with the **Hint** button or `Ctrl + H`. **Show Solution** loads a reference solution straight into the editor when you would rather read the answer and understand it.

Seek & Destroy

SQL Performance Playground — tune real T-SQL against a live SQL Server engine

Overall progress

1/80

Settings

Search lessons (title, topic)...

BEGINNER

1/20

The fundamentals of making SQL Server fast: how indexes turn scans into seeks, why wrapping a column in a function or the wrong data type quietly disables an index, and how to cover, order, and range-search your data. Master these and you'll fix the majority of everyday slow queries.

Table Scan vs. Index Seek

An equality lookup on an unindexed column scans the whole table; a nonclustered index turns the Clustered Index Scan into a fast Index Seek. Why it matters: this is the single most common cause of a "why is the app suddenly slow" ticket — and usually a five-minute fix once you know what to check.

~10m ★★★★★ · best 133 reads

Sargable Predicates: Keep the Colu...

Wrapping a column in a function makes a predicate non-sargable and forces a scan; keep the column bare so the index can seek. Why it matters: wrapping a column in a function is one of the most common, hardest-to-spot ways developers accidentally disable an index they assume is working.

~12m ★★★★★

Implicit Conversions Kill Index Seeks

Comparing an indexed string column to a number makes SQL Server convert every row and abandon the index; match the literal's type to keep the seek. Why it matters: type mismatches between application code and the database schema are extremely common, (especially with ORMs) and silently defeat indexes without any error or warning.

~12m ★★★★★

ORDER BY Without an Index Adds a ...

Table Scan vs. Index Seek

Indexing · plan reading · ~10 min ★★★★★ · solved

Sarge — AI Tutor

Ask Sarge about this lesson — how it works, why it happens, or what to try.

Ask the tutor...

Send

LESSON

What This Lesson Is About

This lesson is about the single most common performance problem in any database: asking for a few rows but making the engine look at *all* of them. You'll see the difference between a **table scan** (checking every row, one by one) and an **index seek** (jumping straight to the rows you need).

Why does this happen? A table on its own is just a big pile of rows with no particular fast way to find "the ones where CustomerId = 42." Unless you build a helper structure, the database has no shortcut — it has to open every page and check every row itself.

Here's the mental model: think of a huge phone book with no alphabetical order — names just written down in the order people signed up. If you want everyone named "Smith," you have to read every single page, front to back. That's a table scan.

Now imagine someone re-sorts a copy of that phone book by last name and tapes labels on it. Suddenly you can flip straight to the "S" section. That's what an **index** does — it's a separate, sorted copy of some columns that lets SQL Server jump straight to the matching rows instead of reading everything. This lesson shows you how to spot the "reading everything" problem in a query plan, and how adding the right index turns a slow scan into a fast, targeted seek.

Run

Reset Lesson

Hint (2/3)

Show Solution

1 SELECT COUNT(\*) FROM Orders;

2 SELECT \* FROM Orders WHERE CustomerId = 42;

Results

Statistics

Execution Plan

Messages

Scratch run — not graded

| OrderId | CustomerId | OrderDate                   | Total | Status  |
|---------|------------|-----------------------------|-------|---------|
| 3       | 4          | 2024-12-29T00:00:00.0000000 | 4.5   | Open    |
| 5003    | 4          | 2024-04-18T00:00:00.0000000 | 4.5   | Closed  |
| 10003   | 4          | 2024-08-06T00:00:00.0000000 | 4.5   | Shipped |
| 15003   | 4          | 2024-11-24T00:00:00.0000000 | 4.5   | Open    |
| 20003   | 4          | 2024-03-14T00:00:00.0000000 | 4.5   | Closed  |
| 25003   | 4          | 2024-07-02T00:00:00.0000000 | 4.5   | Shipped |
| 30003   | 4          | 2024-10-20T00:00:00.0000000 | 4.5   | Open    |
| 35003   | 4          | 2024-02-08T00:00:00.0000000 | 4.5   | Closed  |
| 40003   | 4          | 2024-05-28T00:00:00.0000000 | 4.5   | Shipped |
| 45003   | 4          | 2024-09-15T00:00:00.0000000 | 4.5   | Open    |

© 2026 Johan Potgieter

Privacy

Terms

GitHub

Built with React + ASP.NET Core

Sarge, the built-in AI tutor, answers questions in the context of the current lesson.

**Sarge** is an AI tutor that knows which lesson you are on. Ask why a plan chose a scan, or what a warning means. It is optional: without an API key configured, the panel explains how to set one up and the rest of the app is unaffected.

## 8. Concurrency lessons

The screenshot displays the 'Seek & Destroy' SQL Performance Playground interface. On the left, a sidebar contains a search bar, settings, and a list of lessons. The main area shows a lesson titled 'Blocking: A Reader Stuck Behind an Uncommitted Writer' with a description, a 'What This Lesson Is About' section, and a 'Why this happens in real databases' section. The bottom right features a 'Run Interleaving' button and a timeline view showing the execution of two sessions, Session A and Session B, with their respective SQL queries and execution times.

**Seek & Destroy**  
SQL Performance Playground — tune real T-SQL against a live SQL Server engine

Overall progress: 1/80

**Settings**  
blocking

**ADVANCED** 0/5  
Where the optimizer and the engine get subtle: parameter sniffing, state statistics, tipping points, heaps and RID lookups, anti-joins, catch-all queries, and a full arc of concurrency — blocking, deadlocks, lock escalation, isolation levels, and snapshot conflicts — validated against a real two-session engine.

**Blocking: A Reader Stuck Behind an Ur**  
A reader blocks behind an uncommitted writer's exclusive lock; row-versioning (RCSI) lets it read the last committed version without waiting. Why it matters: blocking is one of the most common production incidents in any transactional system — and one of the hardest to reproduce in testing, since it only bites under real concurrent load.  
concurrency ~15m ★★★★★

**Lock Escalation: One Big Update Loc...**  
A single huge UPDATE escalates to a table lock and blocks an unrelated row; batching it under the escalation threshold keeps locks fine-grained. Why it matters: batch jobs and bulk updates are routine in production, and lock escalation is a common, invisible way they end up blocking completely unrelated user traffic.  
concurrency ~15m ★★★★★

**SERIALIZABLE Key-Range Locks Bloc...**  
A SERIALIZABLE read takes key-range locks that block inserts into that range: use the weakest correct isolation so writers proceed. Why it matters: choosing the right isolation level is a core production skill — using stronger guarantees than a query actually needs is a common cause of unexpected blocking.

**Blocking: A Reader Stuck Behind an Uncommitted Writer**  
concurrency locking blocking isolation ~15 min ★★★★★

Sarge — AI Tutor

LESSON A- 14px A+

**What This Lesson Is About**

Imagine a single-occupant bathroom in an office. If someone walks in and locks the door, everyone else who needs it has to stand outside and wait, no matter how urgent their business is. SQL Server locking works the same way, and “blocking” is what happens when one person waiting behind that locked door is just a paying customer who wanted to peek in and check something quickly, not even use the room themselves.

In everyday terms: this lesson is about what happens when two pieces of work touch the same row of data at the same time — one of them is changing it, the other just wants to look at it. Most people assume that “just reading” is harmless and should never have to wait. In SQL Server’s default configuration, that assumption is wrong, and this lesson is about why.

**Why this happens in real databases.** Any database that lets many users work on the same data at once needs a way to stop them from stepping on each other’s toes. If two people could freely read and write the same row simultaneously with zero coordination, you’d get corrupted results — someone reading half-updated data, or two updates clobbering each other. The traditional fix, used by SQL Server’s default isolation level (called READ COMMITTED, in its classic locking form) is: before you touch a row, you have to get a “lock” on it. Like a key to that bathroom door. There are different flavors of lock. A reader takes what’s called a **shared (S) lock** — think of it as “I’m looking, but so can anyone else who’s just looking.” A writer takes an **exclusive (X) lock** — “I’m changing this, nobody else touches it until I’m done, not even to look.” Shared locks get along fine with other shared locks (many people can peek into the room together, so to speak), but an exclusive lock is incompatible with

Session A holds an uncommitted UPDATE; Session B’s SELECT blocks behind it.

**Session A**

```
1 -- afterMs: 0
2 BEGIN TRAN; UPDATE Accounts SET Balance = Balance + 100 W
3
4 -- afterMs: 200
5 WAITFOR DELAY '00:00:02'; COMMIT;
```

**Session B**

```
1 -- afterMs: 500
2 SELECT AccountId, Owner, Balance FROM Accounts WHERE Acco
```

Run Interleaving

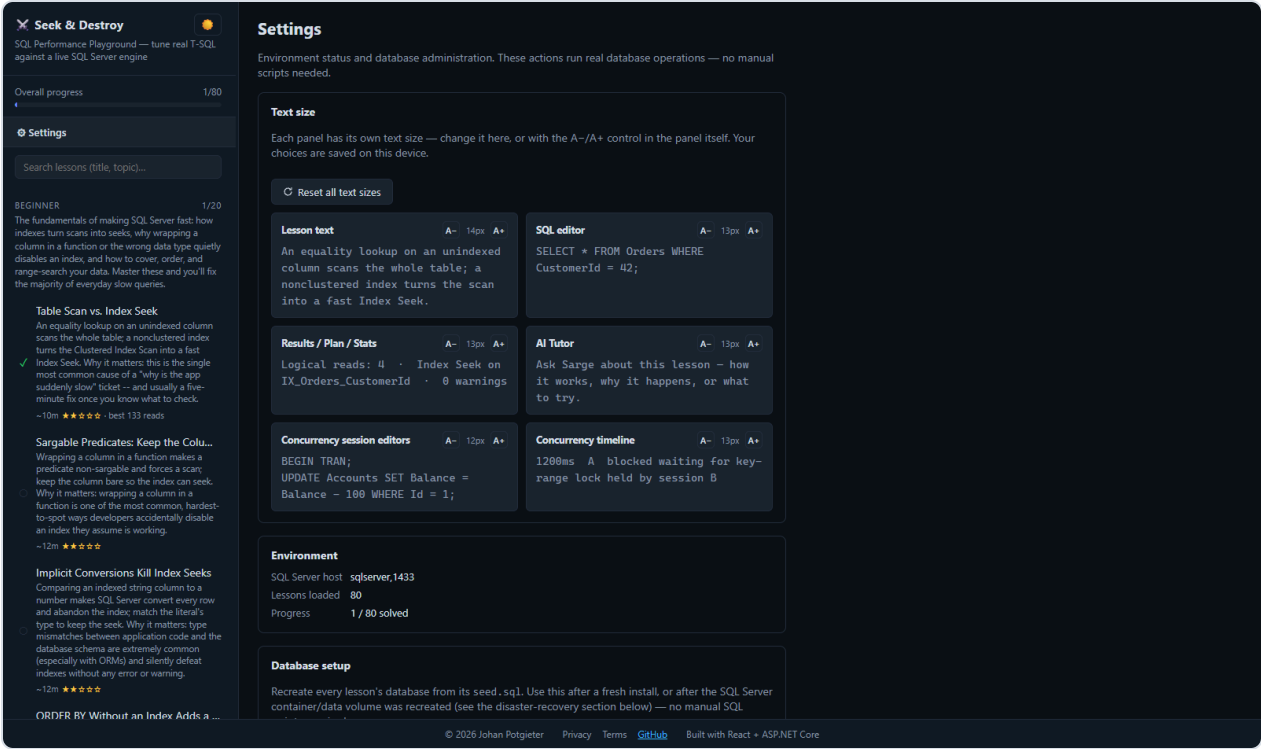
© 2026 Johan Potgieter Privacy Terms GitHub Built with React + ASP.NET Core

A concurrency lesson runs two real sessions against the engine and shows an interleaved timeline.

Some lessons are not about a single slow query but about two sessions fighting. These give you **two editors**, one per session, and run them against the engine in a controlled interleaving. The timeline afterwards shows what actually happened — which session blocked, how long it waited, which lock it wanted, and which session the engine chose as a deadlock victim.

These are graded on outcomes rather than plans: did the deadlock disappear, did both transactions commit, did blocking stay under a threshold. Topics include blocking chains, deadlocks in both directions, lock escalation, key-range locks under `SERIALIZABLE`, and snapshot update conflicts.

# 9. Settings



The Settings page: database maintenance, progress reset and global text size.

| Action                 | What it does                                                                                                                      |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Reset this lesson      | Restores one lesson's database to its original seeded state, undoing any indexes you created.                                     |
| Reseed all databases   | Rebuilds every lesson's data from scratch.                                                                                        |
| Reset progress         | Clears every solved mark and best score. Your lesson data is untouched.                                                           |
| Recreate SQL container | Destroys and rebuilds the SQL Server container entirely — the last resort if the engine gets into a bad state. Local Docker only. |
| Text size              | Sets the font size of each panel independently, with one button to restore all defaults.                                          |

# 10. Customising the workspace

The layout adapts to you and remembers your choices between sessions.

- **Drag the divider** between the lesson text and the editor to change their widths.
- **Drag the top edge of the results pane** up or down to change its height — useful when a result set is long and you want more of it on screen, or when you would rather see more SQL.
- **Text size** is adjustable per panel using the **A-** / **A+** controls, so you can enlarge the lesson text without shrinking the editor.
- **Themes:** the sun/moon button switches between dark and light.

**Seek & Destroy**  
SQL Performance Playground — tune real T-SQL against a live SQL Server engine

Overall progress 1/80

**Settings**  
Search lessons (title, topic)...

**BEGINNER** 1/20  
The fundamentals of making SQL Server fast: how indexes turn scans into seeks, why wrapping a column in a function or the wrong data type quietly disables an index, and how to cover, order, and range-search your data. Master these and you'll fix the majority of everyday slow queries.

**Table Scan vs. Index Seek**  
An equality lookup on an unindexed column scans the whole table; a nonclustered index turns the Clustered Index Scan into a fast Index Seek. Why it matters: this is the single most common cause of a "why is the app suddenly slow" ticket -- and usually a five-minute fix once you know what to check.  
~10m ★★★★★ - best 133 reads

**Sargable Predicates: Keep the Colu...**  
Wrapping a column in a function makes a predicate non-sargable and forces a scan; keep the column bare so the index can seek. Why it matters: wrapping a column in a function is one of the most common, hardest-to-spot ways developers accidentally disable an index they assume is working.  
~12m ★★★★★

**Implicit Conversions Kill Index Seeks**  
Comparing an indexed string column to a number makes SQL Server convert every row and abandon the index; match the literal's type to keep the seek. Why it matters: type mismatches between application code and the database schema are extremely common (especially with ORMs) and silently defeat indexes without any error or warning.  
~12m ★★★★★

**ORDER BY Without an Index Adds a ...**

**Settings**  
Environment status and database administration. These actions run real database operations — no manual scripts needed.

**Text size**  
Each panel has its own text size — change it here, or with the A-/A+ control in the panel itself. Your choices are saved on this device.  
Reset all text sizes

**Lesson text** A- 14px A+  
An equality lookup on an unindexed column scans the whole table; a nonclustered index turns the scan into a fast Index Seek.

**SQL editor** A- 13px A+  
SELECT \* FROM Orders WHERE CustomerId = 42;

**Results / Plan / Stats** A- 13px A+  
Logical reads: 4 · Index Seek on IX\_Orders\_CustomerId · 0 warnings

**AI Tutor** A- 13px A+  
Ask Sarge about this lesson — how it works, why it happens, or what to try.

**Concurrency session editors** A- 12px A+  
BEGIN TRAN;  
UPDATE Accounts SET Balance = Balance - 100 WHERE Id = 1;

**Concurrency timeline** A- 13px A+  
1200ms A blocked waiting for key-range lock held by session B

**Environment**  
SQL Server host: sqlserver,1433  
Lessons loaded: 80  
Progress: 1 / 80 solved

**Database setup**  
Recreate every lesson's database from its seed.sql. Use this after a fresh install, or after the SQL Server container/data volume was recreated (see the disaster-recovery section below) — no manual SQL

© 2026 Johan Potgieter Privacy Terms GitHub Built with React + ASP.NET Core

*The light theme. Every panel, badge and plan colour has a light-mode equivalent.*

The layout is responsive: narrow the window and the lesson text moves above the editor rather than beside it; narrow it further and the lesson list becomes a strip above the workspace.

**Print** exports the current lesson as a clean, printable study reference with the narrative and any hints you have revealed.

## 11. Keyboard shortcuts

| Shortcut         | Action                 |
|------------------|------------------------|
| Ctrl + Enter     | Run the query (graded) |
| Ctrl + H         | Reveal the next hint   |
| Ctrl + Shift + R | Reset the lesson       |

Right-click **Run Selection (not graded)** has deliberately not been given a keyboard shortcut, so that Ctrl + Enter always means the graded run and can never be confused with a scratch run.

## 12. The curriculum

Eighty lessons, twenty per level. Each was validated end to end: the starting query genuinely fails its pass conditions and the reference solution genuinely passes them.

| Level               | What it covers                                                                                                                                         |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Beginner</b>     | Scans versus seeks, sargable predicates, implicit conversions, sorting, covering indexes, composite keys, range and date predicates.                   |
| <b>Intermediate</b> | Covering and filtered indexes, key column order, keyset pagination, conversions across joins, EXISTS versus COUNT, accidental cross joins.             |
| <b>Advanced</b>     | Blocking, deadlocks, tempdb spills, stale statistics, parameter sniffing, the tipping point, lock escalation, isolation levels, catch-all queries.     |
| <b>Expert</b>       | Scalar UDF inlining, columnstore, partition elimination, forwarded records, compression, window-frame costs, memory-grant spills, residual predicates. |

## 13. Troubleshooting

---

| Symptom                                       | What to do                                                                                                                    |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Lessons do not load                           | Check the API is healthy at <code>/api/health</code> ; it should report 80 lessons.                                           |
| A lesson behaves oddly after your experiments | Use <b>Reset Lesson</b> , which restores that lesson's data and drops any indexes you added.                                  |
| "Run Selection" is missing from the menu      | Select some SQL first. The item only appears when there is a selection.                                                       |
| A run seems slow the first time               | Some lessons seed large tables — up to 1.2 million rows — on first use. Later runs are fast.                                  |
| A columnstore lesson will not work            | Columnstore needs a SQL Server tier that supports it. Those lessons are flagged in the app and run best locally under Docker. |